

Optimización de Agentes de Lenguaje de Código Abierto: Un Estudio de Caso sobre Latencia y Gestión de Estado con Gemma

Autora:
Ing. Lesly Zerna
Investigadora Independiente |
Google Developer Expert

Presentado en la Second South American NLP School (3-4 de Agosto, 2026 — Buenos Aires, Argentina)

El Problema

Del Prototipo Tutorial a la Arquitectura en Producción

Cuello de Botella: La acumulación lineal del historial (naive context stacking) provoca un crecimiento ilimitado de la ventana de contexto.

Impacto: En modelos locales, esto genera saturación de memoria, riesgo de exceder el límite de tokens de entrada y degradación en la calidad de respuesta (Lost in the Middle).

Metodología y Entorno

Arquitectura y Metodología de Benchmark

- Modelo & Entorno:** Gemma 2 (2B) ejecutado de forma local mediante Ollama.
- Metodología:** Resultados promediados sobre N=3 corridas independientes, con función de warm-up previa para eliminar la distorsión por carga en frío del modelo.
- Conteo de Tokens:** Medición de tokens de entrada reales mediante el parámetro `prompt_eval_count` de la API de Ollama.
- Pipeline B (Optimizado):** Poda del historial (Context Pruning) para acotar el estado antes de la inferencia.
- Nota de Escalabilidad:** Se intentó una comparación con Gemma2 (9B) en hardware de consumo (Mac); tras >30 min sin respuesta completa, se descartó — evidencia directa de la barrera de hardware que motiva el uso de modelos ligeros en LATAM.

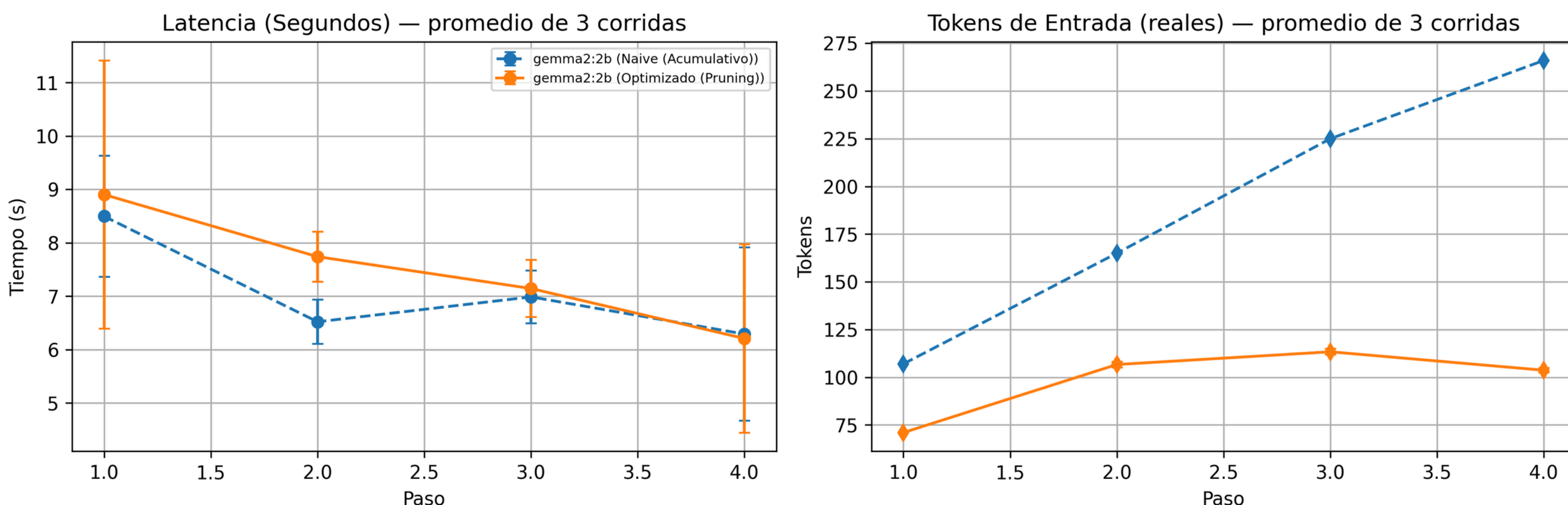


Figura 1. Comparativa de Latencia Promedio (s) y Tokens de Entrada Reales (N=3 corridas con barras de error). En el Pipeline Naive los tokens crecen de 107 a 266 de forma lineal, mientras que el Pipeline Optimizado logra una meseta en ~104 tokens, garantizando la sostenibilidad de la memoria de contexto sin alterar la generación autoregresiva.

Análisis de Resultados

61% de ahorro en tokens de entrada (104 vs. 266 tokens reales en el Paso 4)

- Control de Estado:** El Pipeline Optimizado frena el crecimiento ilimitado de contexto, logrando un ahorro del 61% en tokens de entrada en el cuarto paso.
- Latencia:** En modelos ultra-ligeros (2B), el tiempo total (~6 a 8s) está dominado por la generación autoregresiva de salida, no por el procesamiento del prompt de entrada — la reducción de tokens de contexto no se tradujo en una mejora de latencia estadísticamente clara a esta escala (barras de error solapadas).
- Memoria/Estado:** El control de estado sí previene el colapso de la memoria de contexto a largo plazo, independientemente de su efecto en latencia.
- Trade-off:** La poda agresiva estabiliza el entorno local a cambio de una posible pérdida de contexto histórico lejano.

Conclusiones e Impacto

Soberanía y Accesibilidad en LATAM

- Acceso Soberano:** Se valida la viabilidad de construir agentes de PLN robustos y eficientes utilizando modelos abiertos como Gemma, sin depender de APIs propietarias de pago.
- Relevancia Regional:** Un marco de trabajo ideal para universidades, laboratorios de investigación y empresas en Latinoamérica con restricciones de cómputo local.
- Trabajo Futuro:** Evaluar el mismo pipeline en modelos de mayor escala (Gemma2 9B) sobre infraestructura con GPU dedicada, para determinar si la ventaja de latencia del pruning se hace significativa a mayor tamaño de modelo.

Escanea el código QR para acceder al repositorio en GitHub con el script de benchmark en Python — incluye tanto los resultados exitosos con Gemma2 (2B) como la limitación documentada con Gemma2 (9B).



Contacto:
<https://www.linkedin.com/in/lesly-zerna/>